

[Bài đọc] Nguyên lý thiết kế IoC và kỹ thuật Dependency Injection - DI

1. 🌱 IoC (Inversion of Control) là gì?

- **Inversion of Control (Đảo ngược điều khiển)** là một nguyên lý thiết kế trong lập trình, trong đó việc kiểm soát luồng thực thi hoặc khởi tạo đối tượng không do chính đối tượng quyết định, mà được chuyển giao cho một phần khác của hệ thống
- 📌 Thay vì **tự khởi tạo các đối tượng phụ thuộc**, lớp sẽ nhận các phụ thuộc từ bên ngoài
- Ví dụ truyền thống (không dùng IoC):

```
public class ServiceA {  
    private ServiceB serviceB = new ServiceB(); // Tự tạo đối tượng ServiceB  
}
```

- Khi dùng IoC

```
public class ServiceA {  
    private ServiceB serviceB;  
  
    public ServiceA(ServiceB serviceB) { // Nhận từ bên ngoài  
        this.serviceB = serviceB;  
    }  
}
```

- Trong Spring, phần "bên ngoài" chính là **Spring Container**, nó quản lý và cung cấp các đối tượng khi cần

2. 🧩 Dependency Injection (DI) là gì?

- **Dependency Injection** là kỹ thuật cụ thể để thực hiện nguyên lý IoC. Tức là "**tiêm**" (**inject**) các **phụ thuộc (dependencies)** vào một đối tượng thay vì để nó tự tạo
- Có 3 cách tiêm phụ thuộc (DI):

Cách tiêm	Mô tả
Constructor Injection	Tiêm qua constructor
Setter Injection	Tiêm qua phương thức setter
Field Injection	Tiêm trực tiếp vào trường (dùng <code>@Autowired</code>)

3. 📦 DI hoạt động như thế nào trong Spring?

- Trong Spring Framework:
 - Các đối tượng (bean) được định nghĩa trong **container**
 - Container sẽ **tạo ra và quản lý** các bean
 - Khi một bean cần phụ thuộc khác, container sẽ "**tiêm**" đối tượng phụ thuộc vào

- Ví dụ:

```
@Component
public class BookService {
    private BookRepository bookRepo;

    // Constructor injection
    @Autowired
    public BookService(BookRepository bookRepo) {
        this.bookRepo = bookRepo;
    }
}
```

- Spring sẽ:
 - Tạo bean cho **BookRepository**
 - Tạo bean cho **BookService**

- Tự động truyền **BookRepository** vào **BookService**

4. 🚀 Lợi ích của IoC và DI

- ✅ Giảm phụ thuộc cứng (tight coupling)
- ✅ Dễ bảo trì và thay đổi (loose coupling)
- ✅ Dễ dàng kiểm thử (unit test)
- ✅ Tăng khả năng tái sử dụng
- ✅ Tăng tính module hóa

5. 🗨️ So sánh IoC và DI

Tiêu chí	IoC	DI
Bản chất	Nguyên lý thiết kế	Kỹ thuật triển khai IoC
Mục tiêu	Đảo ngược quyền kiểm soát đối tượng	Cung cấp phụ thuộc từ bên ngoài
Vai trò	Tổng quát	Cụ thể
Spring hỗ trợ	Có	Có (là phần chính trong Spring)

6. 🛠️ Các chú thích (annotation) thường dùng trong DI - Spring

Annotation	Mục đích
@Component	Đánh dấu 1 class là bean
@Service, @Repository, @Controller	Biến thể chuyên biệt của @Component
@Autowired	Tiêm phụ thuộc tự động
@Qualifier	Phân biệt khi có nhiều bean cùng kiểu
@Configuration	Khai báo class cấu hình
@Bean	Tạo bean thủ công trong Java Config

7. 📋 Kết luận

- **IoC** là nguyên lý giúp **đảo ngược vai trò điều khiển**, giúp code mềm dẻo hơn
- **DI** là kỹ thuật triển khai IoC hiệu quả nhất, giúp tăng tính mở rộng, kiểm thử và bảo trì
- Spring là framework hàng đầu **tích hợp IoC/DI Interface**

Tài nguyên đọc

thêm: <https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>